

Name / Reg No: _____

Q1.

a) Error Identification

Line 4, for(i = 0; i <= n; i++) - the condition should be i < n. With <= it goes one step too far and reads arr[n] which doesn't exist for this array.

Line 10, int arr[5] = {1,2,3,4,5} - no semicolon at the end.

Line 12, total = sumArray(arr) - the function needs two arguments (arr and n) but only one is passed here.

Line 14, printf("Correct Sum\n") - again, semicolon missing.

Line 17, int c = a / b - b is 0 so this divides by zero.

Line 18-19, int *p; *p = 20; - p was never pointed to anything valid before being used, so this writes into random memory.

b) Classification of Errors

Line 4 - logical error, wrong loop bound.

Line 10 - syntax error.

Line 12 - semantic error (wrong number of arguments).

Line 14 - syntax error.

Line 17 - runtime error, div by zero.

Line 18-19 - runtime error, uninitialised pointer.

c) Compilation Behavior

The compiler will catch the two missing semicolons and the argument mismatch during compilation itself, because these break the grammar/signature rules it checks before generating code.

The division by zero, the bad pointer write and the actual effect of the wrong loop bound only show up when the program is actually run - compilation finishes fine but the program misbehaves or crashes at that point.

Also once a semicolon is missing the parser can get confused about where that line really ends, so it might read the next line as part of the same statement. That can hide or wrongly report whatever semantic/logical issue comes after it, until the semicolon is put back.

d) Code Correction

```
#include <stdio.h>

int sumArray(int arr[], int n) {
    int i, sum = 0;
    for(i = 0; i < n; i++) {
        sum = sum + arr[i];
    }
    return sum;
}

int main() {
    int arr[5] = {1,2,3,4,5};
    int total;
    total = sumArray(arr, 5);
    if(total == 15) {
        printf("Correct Sum\n");
    }
    int a = 10, b = 2;
```

```

    int c = a / b;
    int val = 20;
    int *p = &val;
    *p = 20;
    return 0;
}

```

e) Impact Analysis

The loop bound mistake (line 4) makes the program read outside the array - so total can come out wrong, or crash on unlucky memory layouts.

Division by zero (line 17) is one of the most common runtime crashes in C, the program just stops with an arithmetic exception.

Writing through the uninitialised pointer *p* is worse in a way, because it doesn't always crash - sometimes it silently overwrites something else in memory and the bug shows up somewhere completely unrelated later.

In general - bad pointer use corrupts memory it doesn't own, dividing by zero kills the program instantly, and an infinite loop just freezes the program forever since it never reaches the return statement.

f) Advanced Compiler Perspective

Compilers could say exactly what's wrong and where instead of a generic "error" - e.g. name the function, the expected argument count, the line.

Two things that would help: (1) static analysis that tracks whether a pointer was assigned before it's dereferenced, and flags it if not, and (2) a debug-mode check on array indexing so an out-of-range access throws a clear message instead of just reading garbage.

Q2.

a) Error Identification

Line 3 - 1num, starts with a digit.

Line 4 - rate@, @ isn't allowed in a name.

Line 5 - 'abc', a char constant can only hold one character.

Line 6 - 50\$20, \$ is not a valid character in C at all.

Line 7 - num#1, # not allowed.

Line 8 - %count, can't start a name with %.

Line 9 - the string "Welcome to Compiler Design is never closed with a quote.

Line 10 - 09, 9 isn't a valid octal digit so this constant is invalid.

Line 11 - 0xZ12, Z isn't a hex digit.

Line 12 - 12.3.4, two decimal points.

Line 13 - "Hello is missing its closing quote too.

Line 16 - invalid-id, hyphens aren't allowed inside identifiers.

Line 17 - 5..6, two dots in a row.

Line 18 - 10@5, @ again.

Line 19 - 'xy', more than one character in a char constant.

Line 20 - 3value, starts with a digit.

b) Error Classification

Identifier rule broken - lines 3, 4, 7, 8, 16, 20. A name has to start with a letter or underscore, and after that only letters/digits/underscore are allowed.

Constant formation broken - lines 6, 10, 11, 12, 17, 18. Numbers have their own grammar for decimal/octal/hex/float and none of these follow it.

String/char literal rule broken - lines 5, 9, 13, 19. A char literal must be exactly one character, a string must close with a matching quote.

All of these get caught at the lexical stage since the scanner can't even form a valid token out of them - it's before parsing or meaning comes into play.

c) Token Correction

1num -> num1 rate@ -> rate 'abc' -> 'a' 50\$20 -> 5020
num#1 -> num1 %count -> count "Welcome to Compiler Design) -> "Welcome to Compiler Design"
09 -> 9 0xZ12 -> 0x112 12.3.4 -> 12.34 "Hello -> "Hello"
invalid-id -> invalid_id 5..6 -> 5.6 10@5 -> 105 'xy' -> 'x' 3value -> value3

d) Compiler Behavior Analysis

The lexical analyser goes through the source character by character, trying to match the longest valid pattern against its token rules (identifiers, numbers, strings, operators etc). When nothing matches, that's a lexical error.

It doesn't just stop there though - most compilers use what's called panic-mode recovery, meaning they skip past the bad characters until they hit something recognisable (space, semicolon, quote) and keep scanning. So it keeps going and reports several errors in one pass instead of dying on the first one.

e) Advanced Thinking

A rough DFA for identifiers: start state -> on letter/underscore go to an "accepting" state that loops on letters/digits/underscore. If the very first character is a digit, send it to a separate dead/error state instead - that traps anything starting with a number before it's even treated as an identifier.

For beginners the error message should point at the exact spot and explain in plain words, something like: "1num isn't a valid name - names can't start with a digit, try num1 instead", rather than just "lexical error".

Q3.

a) Identify Errors

Line 3 - return a / b, missing semicolon.

Line 6 - int #value = 20, # not allowed in identifier.

Line 7 - float num = 15.5, missing semicolon.

Line 9 - divide(10), function needs 2 args, only 1 given.

Line 10 - if(result = 5), this is assignment not comparison, so the condition is always true and result gets overwritten too.

Line 11 - printf("Result is 5\n"), missing semicolon.

Line 15 - int z = x / y, y is 0.

Line 17 - arr[4], the array only has indices 0 to 3.

Line 18-19 - ptr is dereferenced without ever being initialised.

Line 21 - total = x + y * result + 0, the + 0 does nothing, it's just wasted computation.

Line 22-24 - while(x < 20) never updates x, so it never ends.

b) Classify by Compiler Phase

Lexical - line 6.

Syntax - lines 3, 7, 11 (missing semicolons).

Semantic - line 9 (arg mismatch), line 10 (= used where == was meant).

Intermediate code issue - line 10, because the wrong operator changes what value actually flows into the generated code, not what was intended.

Optimization issue - line 21, the redundant +0 should be stripped out by the optimizer.

Runtime errors - line 15 (div by 0), line 17 (array out of bounds), line 18-19 (bad pointer), line 22-24 (infinite loop).

c) Compilation vs Execution

Caught at compile time: lines 3, 6, 7, 11 (the lexical/syntax ones) and line 9, since divide's signature is known and the call doesn't match it.

Only shows up at runtime: line 15, 17, 18-19, and 22-24 (the program just hangs). Line 10 technically compiles fine too, its real problem only shows once it runs.

Same reasoning as Q1 - a missing semicolon confuses the parser about where the statement ends, so it can misread what follows and either report the wrong error or fail to reach the real semantic checks until that's fixed.

d) Correct the Program

```
#include <stdio.h>

int divide(int a, int b) {
    return a / b;
}

int main() {
    int value = 20;
    float num = 15.5;
    int result;
    result = divide(10, 2);
    if(result == 5) {
        printf("Result is 5\n");
    }
    int x = 10;
    int y = 1;
    int z = x / y;
    int arr[4] = {1,2,3,4};
    printf("%d\n", arr[3]);
    int val = 25;
    int *ptr = &val;
    *ptr = 25;
    int total;
    total = x + y * result;
    while(x < 20) {
        printf("%d\n", x);
        x++;
    }
    return 0;
}
```

e) Deep Analysis

Bad pointer usage: *ptr = 25 with an uninitialised ptr writes to whatever address happens to be sitting in memory - could be unused space (crash), could be something that actually matters (silent corruption that shows up as a totally different bug later).

Array bounds: arr[4] reads past the last valid element. That spot in memory might belong to some other variable, so you get a garbage value back, and if it were a write instead of a read it could quietly overwrite that variable.

Precedence/intermediate code: if an expression is evaluated in the wrong order at that stage, the three-address code that comes out doesn't match what the source actually meant - and there's no error to warn about it, the program just gives the wrong number.

f) Advanced Task

1) Catch = used inside a condition automatically at compile time (most compilers already warn for this, it should really be enforced more strongly for beginners).

2) Add optional runtime checks in debug builds, similar to AddressSanitizer, so out-of-bounds access and bad pointer writes get reported right away with the exact location instead of failing silently.

For the optimizer - constant folding and dead code elimination would catch things like $+ 0$ or $* 1$ automatically and remove them before final code is generated.